Uniwersytet Łódzki
Wydział Matematyki i Informatyki

Temat ćwiczenia:

Drzewa Decyzyjne

Sprawozdanie z przedmiotu: ***Zaawansowane Metody Eksploracji***
Studia stacjonarne 2-go stopnia
Kierunek: *Analiza Danych, sem. II*
Prowadzący: *prof. dr hab. inż. Grzegorz Dudek*

Data wykonania ćwiczenia:	26.03.2025
Data złożenia sprawozdania:	09.04.2025
Data zaliczenia:	

Wykonali:	Ocena:
Dawid Sporysiak	

Nr sekcji: **3.19**

A. Cel ćwiczenia

Celem ćwiczeń jest obróbka danych do ich odpowiedniego użytkowania, wyświetlania i interpretowania, trenowanie stosowania modeli uczenia przy użyciu drzewa, lasu, oraz boostingu, oraz stosowanie metody crosswalidacji i używania macierzy pomyłek.

Teoria dotycząca sprawozdania

Drzewa decyzyjne to struktury przypominające schemat blokowy, w których każdy węzeł odpowiada pytaniu dotyczącemu jednej z cech , a każda gałąź jednej z odpowiedzi (lub zakresów wartości). Algorytmy budujące drzewa wybierają cechy maksymalizujące czystość podziałów. Końcowe liście drzewa zawierają przewidywania klasę lub wartość. Drzewa są łatwe do interpretacji, ale podatne na przeuczenie.

Lasy losowe to metoda zespołowa, która łączy wiele drzew decyzyjnych w jeden silniejszy model. Każde drzewo trenowane jest na losowej próbce danych, a przy każdym podziale wybierany jest losowy podzbiór cech. Dzięki temu lasy losowe są bardziej odporne na przeuczenie niż pojedyncze drzewa. W klasyfikacji stosuje się głosowanie większościowe, a w regresji średnią z prognoz.

Boosting to metoda, w której budujemy wiele prostych modeli, ale każdy kolejny uczy się na błędach poprzedniego. Czyli jeśli pierwszy model coś pomylił, drugi próbuje to poprawić, trzeci poprawia dalej itd. Na końcu wszystkie modele są łączone. To pozwala osiągać bardzo dobre wyniki, ale taki model łatwo się przeucza, jeśli nie ustawi się odpowiednich parametrów (np. tzw. „learning rate”).

B. Treść zadania

Zadanie 1

Treść:

1. Zaimportuj niezbędne moduły:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn.model_selection as skm

from matplotlib.pyplot import subplots
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import train_test_split
from sklearn.metrics import PredictionErrorDisplay
from imblearn.over_sampling import RandomOverSampler
from sklearn.tree import (DecisionTreeClassifier as DTC,
                          DecisionTreeRegressor as DTR,
                          plot_tree,
                          export_text)
from sklearn.metrics import (accuracy_score,
                             confusion_matrix,
                             ConfusionMatrixDisplay,
                             mean_squared_error)
from sklearn.ensemble import \
     (RandomForestRegressor as RF,
      GradientBoostingRegressor as GBR)
```

Rozwiązanie zadania:

```
#zadanie 1
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn.model_selection as skm
from matplotlib.pyplot import subplots
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import train_test_split
from sklearn.metrics import PredictionErrorDisplay
from imblearn.over_sampling import RandomOverSampler
from sklearn.tree import (DecisionTreeClassifier as
DTC,DecisionTreeRegressor as DTR,plot_tree,export_text)
from sklearn.metrics import
(accuracy_score,confusion_matrix,ConfusionMatrixDisplay,mean_squared_er
ror)
```

```
from sklearn.ensemble import(RandomForestRegressor as
RF,GradientBoostingRegressor as GBR)
import seaborn as sns
```

Zadanie 2

Treść:

2. Wczytaj dane, dokonaj selekcji danych (każdy student tworzy inny zbiór danych, który składa się z wylosowanych wierszy danych oryginalnych).

   Pobierz dane z https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

```
data = pd.read_csv("healthcare-dataset-stroke-data.csv")
```

```
nr_gr = ?
r_k = ?
np.random.seed(int(nr_gr*r_k))

q = data.shape
idx = np.random.permutation(q[0]) #permutacja wierszy
data = data.iloc[idx[0:5000],:] #selekcja 5000 losowych wierszy

data.head()
```

gdzie za `nr_gr` wstaw numer swojej sekcji a za `r_k` aktualny rok kalendarzowy.

Rozwiązanie zadania:

```
#zadanie 2
data = pd.read_csv("healthcare-dataset-stroke-data.csv")
2
nr_gr = 3.19
r_k = 2025
np.random.seed(int(nr_gr*r_k))
q = data.shape
idx = np.random.permutation(q[0]) #permutacja wierszy
data = data.iloc[idx[0:5000],:] #selekcja 5000 losowych wierszy
data.head()
```

Wynik:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
2286	2860	Male	55.0	0	0	Yes	Private	Rural	82.88	29.4	Unknown	0
2016	25919	Male	48.0	1	0	Yes	Self-employed	Urban	83.34	49.3	never smoked	0
539	40622	Female	43.0	0	0	Yes	Private	Rural	80.83	51.5	Unknown	0
3908	11208	Female	2.0	0	0	No	children	Rural	70.25	17.0	Unknown	0
3033	21192	Female	78.0	0	0	Yes	Private	Urban	93.15	23.6	Unknown	0

Wniosek:

Wymagane było pobranie ze strony https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset?resource=download folderu zip, rozpakowanie go, a następnie zaimportowaniu potrzebnego pliku.

Zadanie 3

Treść:

3. Scharakteryzuj dane (przydatne funkcje: `data.describe()`, `data.info()`).

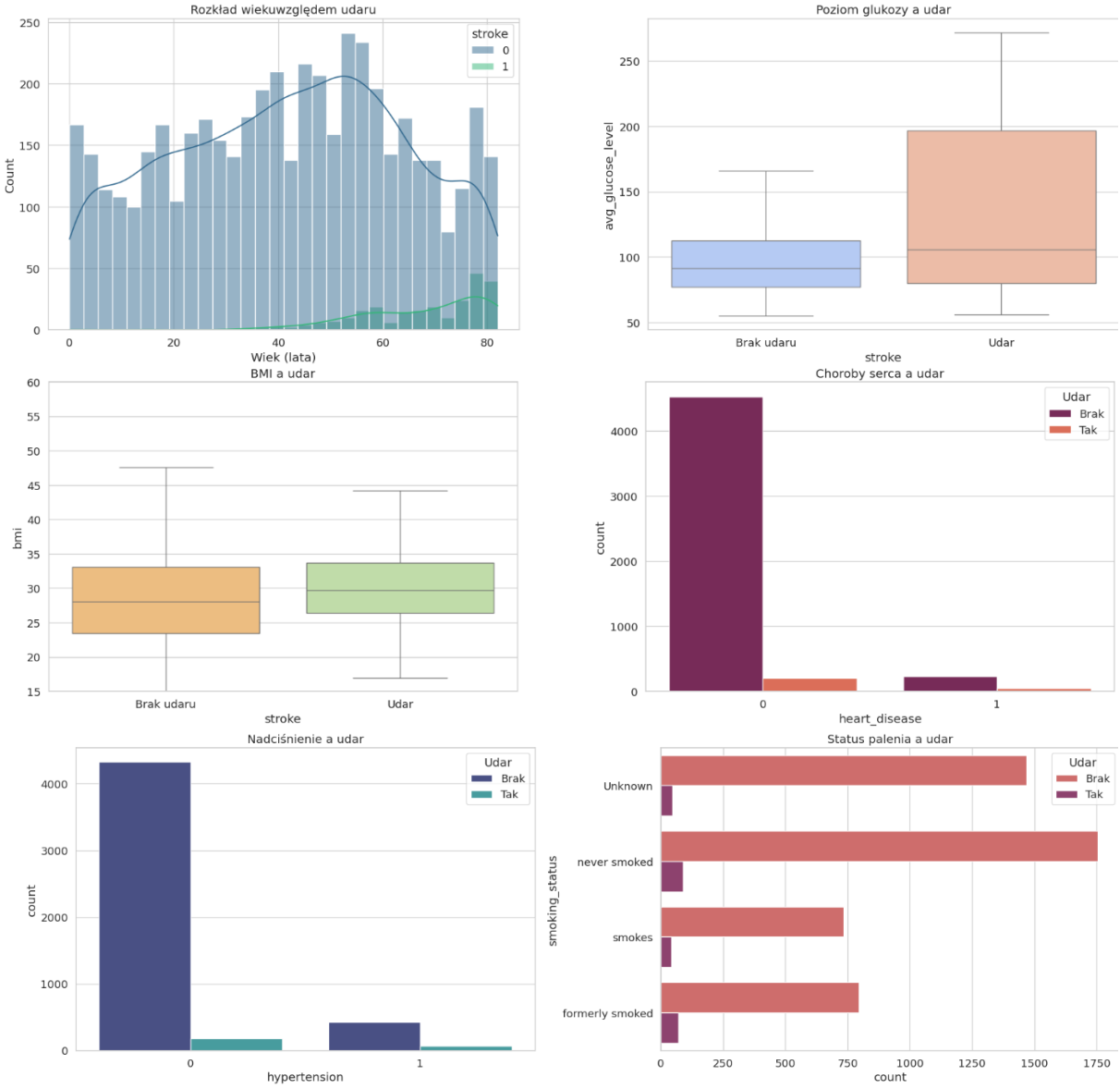
Rozwiązanie zadania:

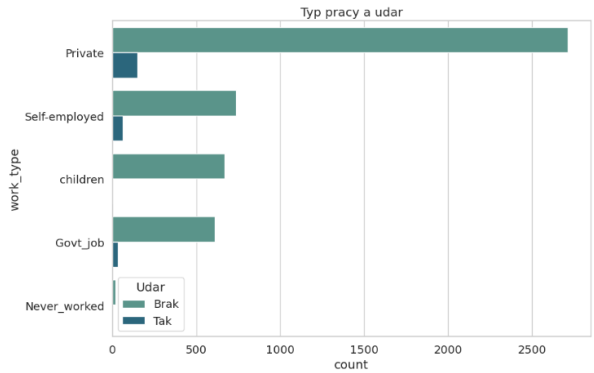
```
sns.set(style="whitegrid", font_scale=1.2)
plt.figure(figsize=(20, 25))
#1. Wiek
plt.subplot(4, 2, 1)
sns.histplot(data=data, x='age', hue='stroke', kde=True, bins=30,
palette="viridis")
plt.title('Rozkład wiekuwzględem udaru')
plt.xlabel('Wiek (lata)')
#2. Poziom glukozy
plt.subplot(4, 2, 2)
sns.boxplot(data=data, x='stroke', y='avg_glucose_level',
showfliers=False, palette="coolwarm")
plt.title('Poziom glukozy a udar')
plt.xticks(ticks=[0, 1], labels=['Brak udaru', 'Udar'])
```

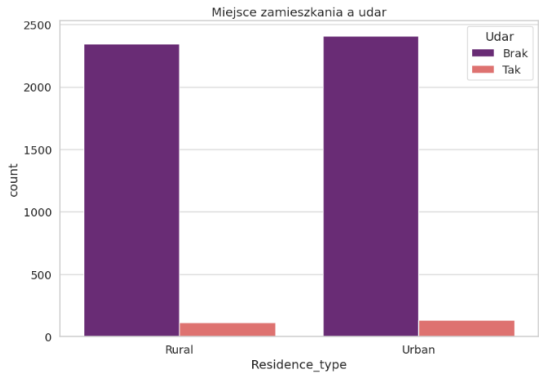
```
#3. BMI
plt.subplot(4, 2, 3)
sns.boxplot(data=data, x='stroke', y='bmi', showfliers=False,
palette="Spectral")
plt.title('BMI a udar')
plt.ylim(15, 60)
plt.xticks(ticks=[0, 1], labels=['Brak udaru', 'Udar'])
#4. Choroby serca
plt.subplot(4, 2, 4)
sns.countplot(data=data, x='heart_disease', hue='stroke',
palette="rocket")
plt.title('Choroby serca a udar')
plt.legend(title='Udar', labels=['Brak', 'Tak'])
#5. nadciśnienie
plt.subplot(4, 2, 5)
sns.countplot(data=data, x='hypertension', hue='stroke',
palette="mako")
plt.title('Nadciśnienie a udar')
plt.legend(title='Udar', labels=['Brak', 'Tak'])
#6. Status palenia
plt.subplot(4, 2, 6)
sns.countplot(data=data, y='smoking_status', hue='stroke',
palette="flare")
plt.title('Status palenia a udar')
plt.legend(title='Udar', labels=['Brak', 'Tak'])
#7. Typ pracy
plt.subplot(4, 2, 7)
sns.countplot(data=data, y='work_type', hue='stroke', palette="crest")
plt.title('Typ pracy a udar')
plt.legend(title='Udar', labels=['Brak', 'Tak'])
#8. miejsce zamieszkania
plt.subplot(4, 2, 8)
sns.countplot(data=data, x='Residence_type', hue='stroke',
palette="magma")
plt.title('Miejsce zamieszkania a udar')
plt.legend(title='Udar', labels=['Brak', 'Tak'])

plt.tight_layout()
plt.show()
plt.figure(figsize=(12, 8))

# Wybieramy tylko kolumny numeryczne
numeric_cols = data.select_dtypes(include=['float64', 'int64']).columns
corr = data[numeric_cols].corr()[['stroke']].sort_values(by='stroke',
ascending=False)

sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1,
center=0)
```

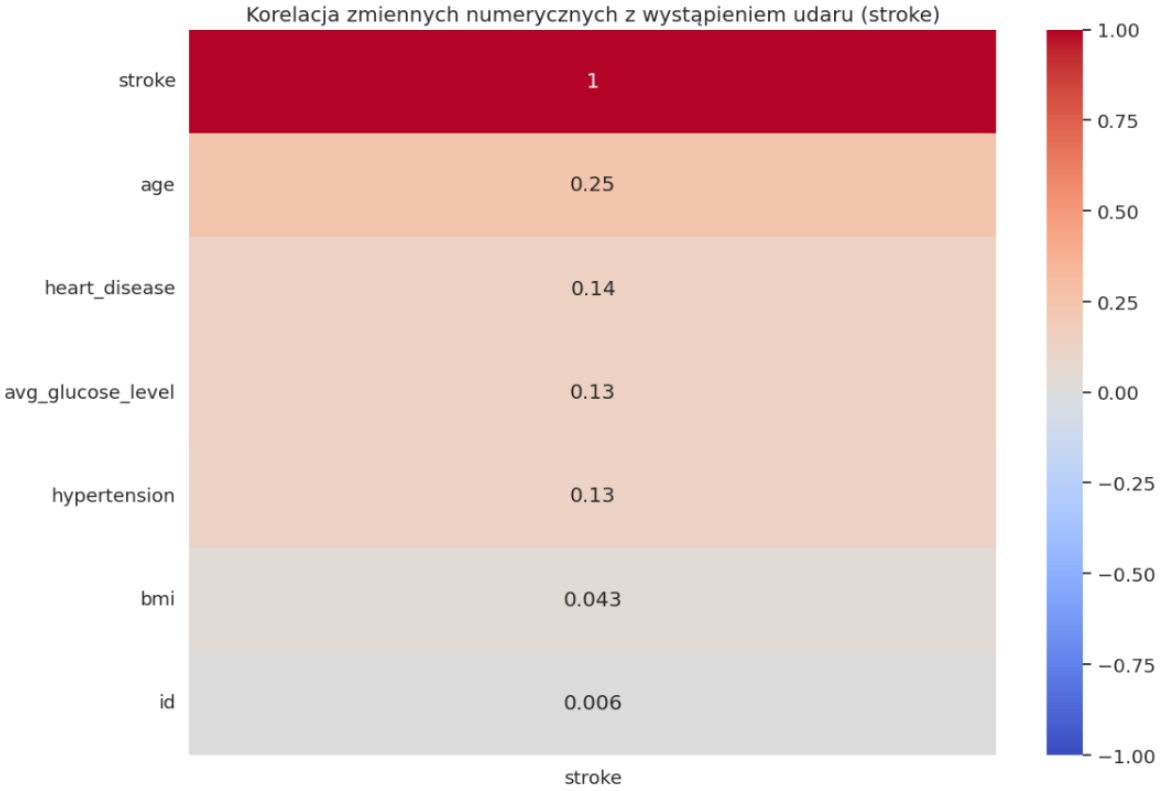
```
plt.title('Korelacja zmiennych numerycznych z wystąpieniem udaru
(stroke)')
plt.tight_layout()
plt.show()
```

Wynik:









Wniosek:

Wszystkie wykresy zostały zrobione dla zmiennych wiek, poziom glukozy, bmi, choroby serca, nadciśnienia, statusu palenia, typu pracy i miejsca zamieszkania

Z wykresu 1 można odczytać, że skłonności do udarów mają starsze osoby

Z wykresu 2 średni poziom glukozy jest zbliżony dla chorych jak i zdrowych, ale dla chorych jesgo górne wartości są znacnznie większe

Wykres 3 pozakuje, że osoby z udarem mają nieco mniejszy rozrzut bmi względem średniej

Wykres 4 pokazuje, że duża większość osób, które było badanych nie miało nigdy chorób serca, osoby które miały udar były w obu grupach, przy tej samej liczebności grup wyniki byłyby zbliżone

Wykres 5 analogicznie jak z nadciśnieniem, lecz tutaj już prawdopodobnie więcej osób chorych miało nadciśnienie

Wykres 6 pokazuje, że dominującą grupą osób są osoby które nigdy nie paliły i osoby o których nie ma informacji odnośnie statusu palenia, w przypadku wybalansowania grup nie widać względnie zależności pomiędzy tymi grupami

Wykres 7 pokazuje, że dzieci bardzo rzadko miewają ataki serca (wykres 1) i pracownicy w firmach mają ich mniej, niż osoby które prowadzą jednoosobowe działalności gospodarcze

Wykres 8 nawiązujący do miejsca zamieszkania również nie wskazuje na zależność między udarem.

Heatmapa zmienych względem udaru pokazuje, że największą zależność ma wiek, następnie nadciśnienie oraz średni poziom glukozy, i małżeństwo

Zadanie 4

Treść:

4. Oczyszczanie danych.

- Usuń nieprzydatną kolumnę 'id' (`data=data.drop('id', axis = 1)`)
- Znajdź i ew. usuń zduplikowane wiersze (`data.duplicated().sum()`)
- Znajdź atrybuty z brakującymi wartościami (`data.isnull().sum()`)
- Usuń wiersze z brakującymi wartościami (`data = data.dropna(axis = 0)`)

Rozwiązanie zadania:

```
#Usunięcie kolumny 'id'
if 'id' in data.columns:
    data = data.drop('id', axis=1)

#Sprawdzenie brakujących wartości
print("Liczba brakujących wartości w kolumnach:\n",
data.isnull().sum())
```

```python
#Usuwanie wierszy z brakującymi danymi (N/A)
initial_rows = len(data)
data = data.dropna(axis=0)
dropped_rows = initial_rows - len(data)
print(f"Usunięto {dropped_rows} wierszy z brakującymi danymi.")

#Sprawdzenie duplikatów
print("Liczba zduplikowanych wierszy przed usunięciem:",
data.duplicated().sum())

#Zduplikowanych wierszy
initial_rows = len(data)
data = data.drop_duplicates()
dropped_duplicates = initial_rows - len(data)
print(f"Usunięto {dropped_duplicates} zduplikowanych wierszy.")

# Wyświetlenie pierwszych wierszy
data.head()
```

Wynik:

```
Liczba brakujących wartości w kolumnach:
 gender                 0
age                    0
hypertension           0
heart_disease          0
ever_married           0
work_type              0
Residence_type         0
avg_glucose_level      0
bmi                  197
smoking_status         0
stroke                 0
dtype: int64
Usunięto 197 wierszy z brakującymi danymi.
Liczba zduplikowanych wierszy przed usunięciem: 0
Usunięto 0 zduplikowanych wierszy.
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
2286	Male	55.0	0	0	Yes	Private	Rural	82.88	29.4	Unknown	0
2016	Male	48.0	1	0	Yes	Self-employed	Urban	83.34	49.3	never smoked	0
539	Female	43.0	0	0	Yes	Private	Rural	80.83	51.5	Unknown	0
3908	Female	2.0	0	0	No	children	Rural	70.25	17.0	Unknown	0
3033	Female	78.0	0	0	Yes	Private	Urban	93.15	23.6	Unknown	0

Wniosek:

W wyniku tego kodu usunięte zostało 197 wierszy, w których brakowało wartości, wszystkie dotyczyły bmi, które czasami było wpisywane jako N/A.

Zadanie 5

Treść:

5. Implementacja drzew decyzyjnych w bibliotece `sklearn` nie akceptuje atrybutów nominalnych. Takie atrybuty należy zamienić na liczbowe (`int64`). W tym celu:

   - Utwórz nową tabelę z danymi (`data2 = pd.DataFrame()`)
   - Przepisz do tej tabeli wszystkie atrybuty liczbowe (np. `data2['age'] = data['age']`)
   - Jeśli atrybut nominalny ma tylko dwie wartości (możesz to sprawdzić funkcją `data.nunique()`, `data.unique()`), zamień te wartości na 0 i 1, np.: `data2['ever_married'] = np.int64(data['ever_married']=='Yes')`
   - Jeśli atrybut nominalny ma *k*>2 wartości, utwórz dla niego *k* nowych atrybutów, np.:

```
data2['work_type_Private'] = np.int64(data['work_type']=='Private')
data2['work_type_Self_emp'] = np.int64(data['work_type']=='Self-employed')
data2['work_type_Govt_job'] = np.int64(data['work_type']=='Govt_job')
data2['work_type_Children'] = np.int64(data['work_type']=='children')
data2['work_type_Never_worked'] = np.int64(data['work_type']=='Never_worked')
```

Rozwiązanie zadania:

```
#Zadanie 5
data2 = pd.DataFrame()
data2['gender']=data['gender']
data2['age']=data['age']
data2['hypertension']=data['hypertension']
data2['ever_married']=data['ever_married']
data2['work_type']=data['work_type']
data2['Residence_type']=data['Residence_type']
data2['avg_glucose_level']=data['avg_glucose_level']
data2['bmi']=data['bmi']
data2['smoking_status']=data['smoking_status']
data2['stroke']=data['stroke']
data2.head()

data2 = pd.DataFrame()
#Kolumny liczbowe
num_cols = ['age','hypertension','avg_glucose_level','bmi','stroke']
for col in num_cols:
    data2[col] = data[col].copy()
#Kolumny binarne
binary_cols = ['ever_married', 'Residence_type', 'gender']
for col in binary_cols:
    if col == 'ever_married':
        data2[col] = np.int64(data[col] == 'Yes')
        #1 = małżeństwo, 0 = brak
    elif col == 'Residence_type':
        data2[col] = np.int64(data[col] == 'Urban')
        #1 = miastowy 0 = spoza miasta
```

```
    elif col == 'gender':
        data2[col] = np.int64(data[col] == 'Male')
        #1 = mężczyzna, 0 = kobieta
#Kolumny z wieloma kategoriami
cat_cols = ['work_type', 'smoking_status']
#Tutaj było wiele wartości zmiennych
data2['work_type_Private'] = np.int64(data['work_type']=='Private')
data2['work_type_Self_emp'] = np.int64(data['work_type']=='Self-
employed')
data2['work_type_Govt_job'] = np.int64(data['work_type']=='Govt_job')
data2['work_type_Children'] = np.int64(data['work_type']=='children')
data2['work_type_Never_worked'] =
np.int64(data['work_type']=='Never_worked')
data2.head()
print("\nTypy danych:\n", data2.dtypes)
```

Wynik:

```
Typy danych:
 age                      float64
hypertension               int64
avg_glucose_level        float64
bmi                      float64
stroke                     int64
ever_married               int64
Residence_type             int64
gender                     int64
work_type_Private          int64
work_type_Self_emp         int64
work_type_Govt_job         int64
work_type_Children         int64
work_type_Never_worked     int64
dtype: object
```

Wniosek:

Typy zostały pozamieniane tak, by mogły one być interpretowalne dla dalszych zadań. Dwu-wartościowe przypadki zostały zamienione na binarne, natomiast te bardziej złożone po prostu na liczby.

Zadanie 6

Treść:

6. Przygotuj dane do uczenia modelu. Utwórz tabele:
   - `X` z przykładami (zawierającą wszystkie kolumny z wyjątkiem `'stroke'`) i
   - `y` z etykietami (kolumna `'stroke'`).

   Podziel dane na część treningową (80% danych) i testową (20%). Przeprowadź próbne uczenie modelu.

```
X = data2.iloc[:, : -1]
y = data2.iloc[:, -1 :]
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2,
random_state = 1)
```

```
dt = DTC(criterion='entropy', max_depth=5, random_state=1)
dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)

accuracy_score(y_test, y_pred)
```

Rozwiązanie zadania:

```
#Wybranie danych, oraz podział na zbiór treningowy i testowy
X = data2.iloc[:, : -1]
y = data2.iloc[:, -1 :]
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size =
0.2,
random_state = 1)
#Badanie max accurancy dla max_depth=20 dla metody crosswalidacji
krzyżowej
dt = DTC(criterion='entropy', max_depth=1, random_state=1)
dt.fit(X_train, y_train)
y_pred = dt.predict(X_test)
accuracy_score(y_test, y_pred)
```

Wynik:

```
0.9927159209157128
```

Wniosek:

Wynik jest bardzo podejrzany. Pochodzi on z niezbalansowania grup między chorymi a zdrowymi osobami wziętymi do testów.

Zadanie 7

Treść:

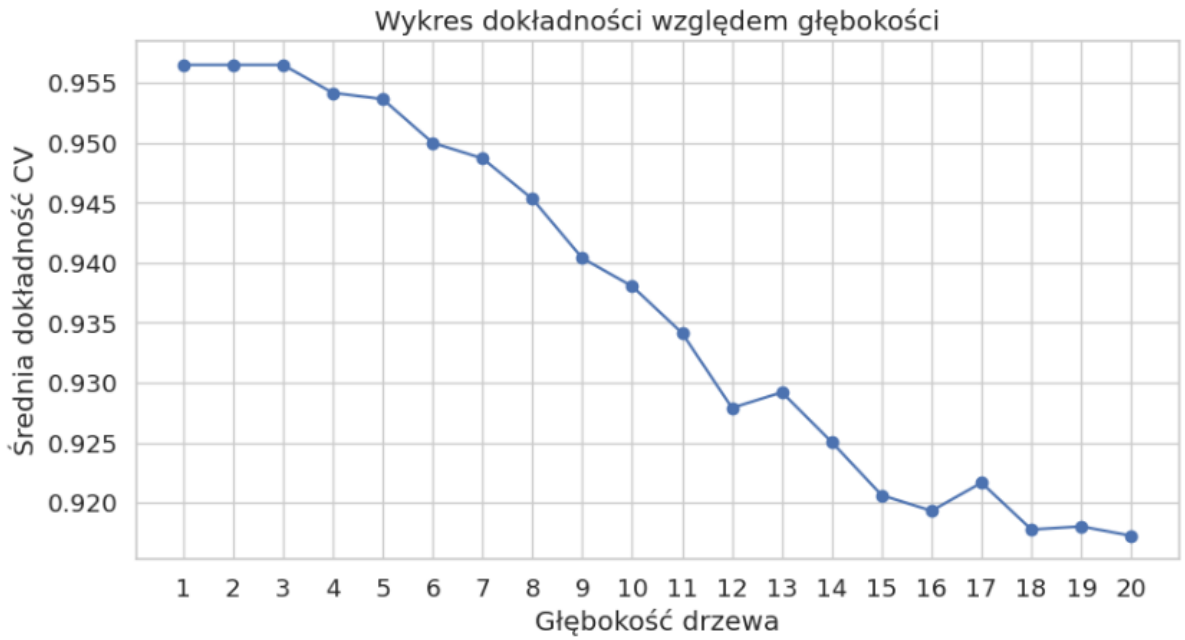
7. W procedurze kroswalidacji znajdź optymalną wartość `max_depth` (wzoruj się na przykładzie podanym na wykładzie – str. 69). Pokaż wykres zależności dokładności klasyfikacji od `max_depth`. Odnotuj dokładność modelu wyznaczoną w kroswalidacji.

Rozwiązanie zadania:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report


X = pd.get_dummies(data2.drop('stroke', axis=1))
y = data2['stroke']

#Podział na zbiór treningowy i testowy
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,random_state=1,stratify=y)
#10-krotna crosswalidacja
max_depths = range(1, 21)
cv_scores = []
for depth in max_depths:
    model = DecisionTreeClassifier(criterion='entropy',
max_depth=depth, random_state=1)
    scores = cross_val_score(model, X_train, y_train, cv=10,
scoring='accuracy')
    cv_scores.append(scores.mean())

#Wykres
plt.figure(figsize=(10, 5))
plt.plot(max_depths, cv_scores, 'bo-')
plt.xlabel('Głębokość drzewa')
plt.ylabel('Średnia dokładność CV')
```

```
plt.title('Wykres dokładności względem głębokości')
plt.xticks(max_depths)
plt.grid(True)
plt.show()
#Najlepsza głębokość
best_depth = max_depths[cv_scores.index(max(cv_scores))]
print(f"Optymalna głębokość: {best_depth}")
print(f"Średnia dokładność CV: {max(cv_scores):.4f}")
```

Wynik:




```
Optymalna głębokość: 1
Średnia dokładność CV: 0.9565
```

Wniosek:

Optymalna głębokość drzewa wynosi 1. Oznacza to, że model jest skrajnie niedouczony, i zakłada bardzo prosty przypadek. Przy większej głębokości drzewa wyniki są wręcz gorsze niż dla tego przypadku, co jest podejrzane.

Dodatkowe sprawdzenie

```python
#Dodatkowe
from sklearn.metrics import accuracy_score
import numpy as np

y_test_array = np.array(y_test).flatten()
stroke_counts = np.bincount(y_test_array)
total_samples = len(y_test_array)

#Accurancy
accuracy = accuracy_score(y_test, y_pred)

print(f"Liczba osób zdrowych (brak udaru): {stroke_counts[0]}")
print(f"Liczba osób chorych (udar): {stroke_counts[1]}")
print(f"Proporcja: {stroke_counts[1]/stroke_counts[0]:.2%} osób miało
udar")


print(f"\nDokładność modelu (accuracy): {accuracy:.2%}")
print(f"Proporcja osób zdrowych w danych testowych:
{stroke_counts[0]/total_samples:.2%}")
```

```
Liczba osób zdrowych (brak udaru): 919
Liczba osób chorych (udar): 42
Proporcja: 4.57% osób miało udar

Dokładność modelu (accuracy): 95.63%
Proporcja osób zdrowych w danych testowych: 95.63%
```

```python
#Accurancy jest całkiem zbliżone do liczby osób zdrowych, wynika to z
niezbalansowania grup, poniższe zadania również będą miały tę wadę
#liczba osób zdrowych w próbie wynosi 95,63%
#W przypadku, gdy w badaniu znacznie przeważa ilość osób zdrowych, w
procesie klasyfikacji najoptymalniej jest założyć, że każda osoba jest
zdrowa.
#Można to porównać do gry w lotto, gdy szansa jest niezwykle mała,
mając nowy los można z niemal 100% dokładnością odpowiedzieć poprawnie
na pytanie, czy los jest wygrany czy przegrany (przegrany).
```

Zadanie 8

Treść:

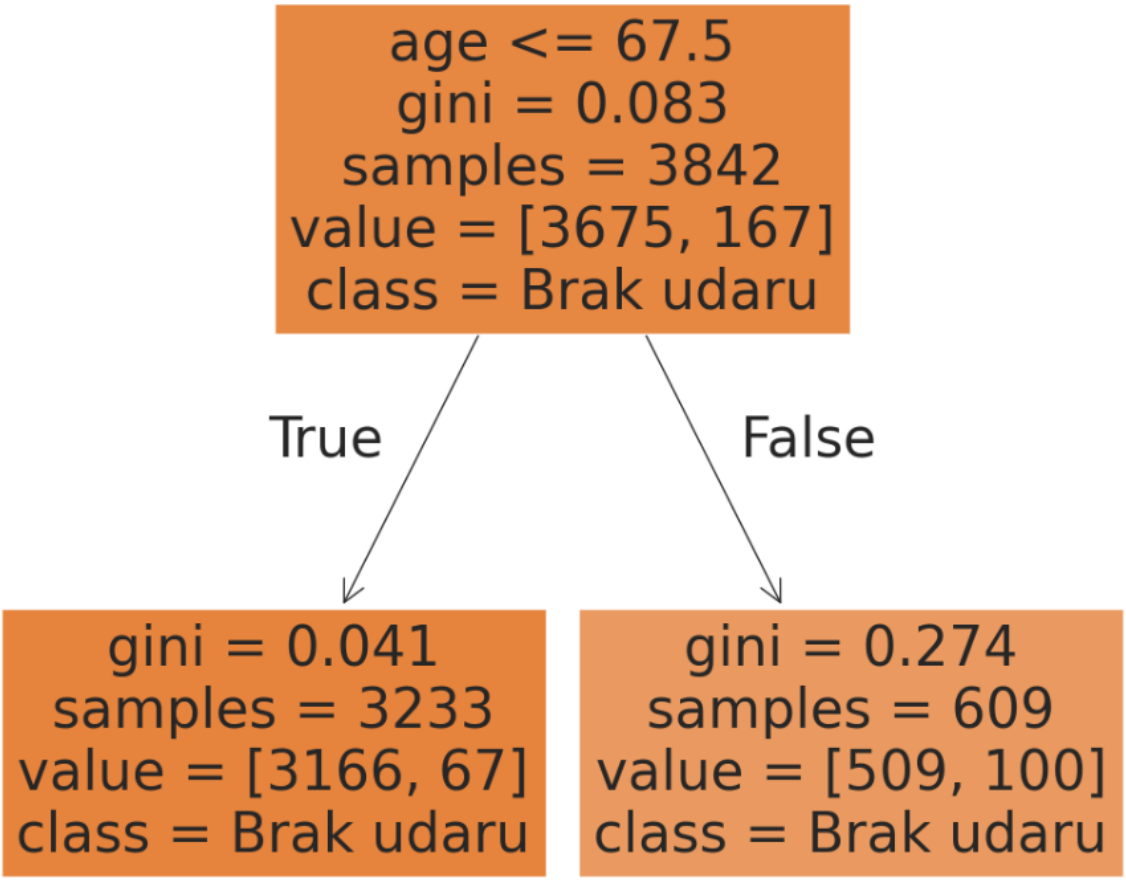
8. Przeprowadź uczenie drzewa na zbiorze treningowym z optymalną wartością hiperparametru `max_depth` znalezioną w p. 7. Oblicz dokładność klasyfikacji dla danych treningowych i testowych. Narysuj drzewo i przedstaw je w postaci reguł decyzyjnych:

```
ax = subplots(figsize=(8,8))[1]
plot_tree(dt,
    feature_names=X.columns,
    ax=ax);

print(export_text(dt,
    feature_names=list(X.columns),
    show_weights=True))
```

Rozwiązanie zadania:

```
#Zadanie 8 bez balansowania
from sklearn.tree import plot_tree, export_text
from matplotlib.pyplot import subplots

#Trenowanie
dt = DecisionTreeClassifier(max_depth=1,random_state=1).fit(X_train,
y_train)

print(f'Dokładność treningowa: {dt.score(X_train, y_train):.4f}')
print(f'Dokładność testowa: {dt.score(X_test, y_test):.4f}')

#Drzewo
fig, ax = subplots(figsize=(12, 12))
plot_tree(dt,feature_names=X.columns,class_names=['Brak udaru',
'Udar'],filled=True,ax=ax)
plt.show()

#Reguły decyzyjne
print(export_text(dt,feature_names=list(X.columns),show_weights=True))
```

Wynik:




```
|--- age <= 67.50
|   |--- weights: [3166.00, 67.00] class: 0
|--- age >  67.50
|   |--- weights: [509.00, 100.00] class: 0
```

Wniosek:

Jest to drzewo decyzyjne z przyciętą głębokością równą 1. Głębokość drzewa jest bardzo niska, dokładność dla większej liczby wręcz maleje (w zadaniu 7 widać na wykresie), zbiór dla tej głębokości jest zdecydowanie niedouczony, i może podawać bardzo sprzeczne wyniki z oczekiwanymi rezultatami.

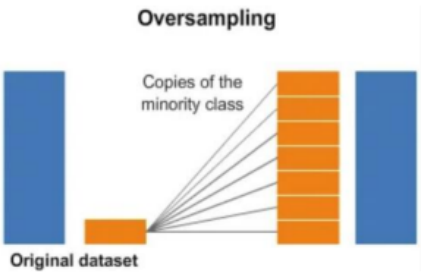
Zadanie 9

Treść:

9. Zwizualizuj wyniki za pomocą `confusion_matrix`.

```
cm = confusion_matrix(y_test, dt.predict(X_test), labels=dt.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=dt.classes_)
disp.plot()
plt.show()
```

Zwróć uwagę, że przykłady z klasy 1 są w większości błędnie klasyfikowane. Pomimo tego dokładność klasyfikatora jest bardzo wysoka. Wynika to z dysproporcji liczebności klas – liczebność klasy 0 przewyższa znacznie liczebność klasy 1. Jest to tzw. problem niezbalansowanych danych (*imbalanced data*). Aby zwiększyć dokładność klasyfikacji dla klasy mniej licznej, rozwiązaniem może być „rozmnożenie" danych treningowych dla tej klasy (*oversampling*).

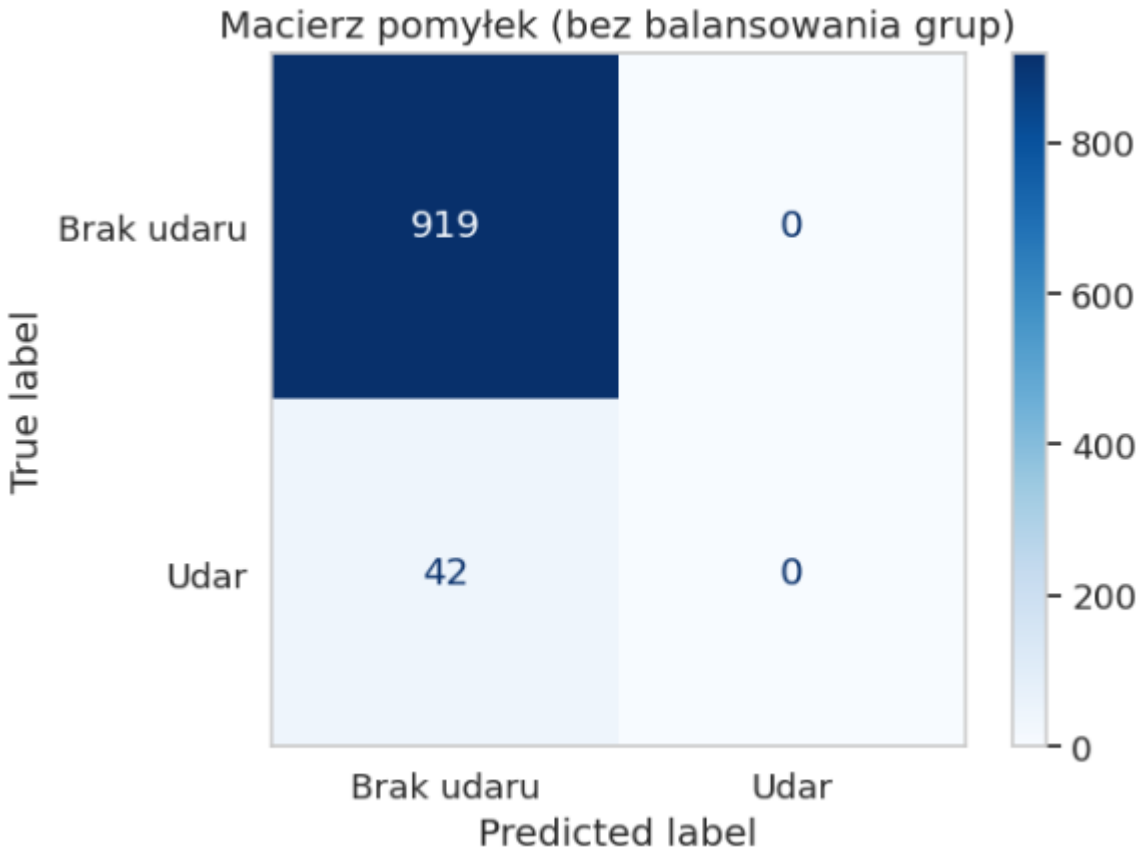



```
ros = RandomOverSampler(random_state = 1)
X_train_resampled, y_train_resampled = ros.fit_resample(X_train, y_train)
```

Rozwiązanie zadania:

```
#zadanie 9 bez balansowania
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

#Macierz pomyłek
cm = confusion_matrix(y_test, dt.predict(X_test))
plt.figure(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Brak
udaru', 'Udar'])
disp.plot(cmap='Blues', values_format='d')
plt.title('Macierz pomyłek (bez balansowania grup)')
plt.grid(False)
plt.show()
```

Wynik:



Wniosek:

Z macierzy pomyłek możemy odczytać, że na 961 przypadków, 919 osób bez udaru zostało poprawnie zaklasyfikowanych do tej grupy, natomiast 42 które miały udar zostały zaklasyfikowane do osób zdrowych. Jest to bardzo złe, ponieważ w takim przypadku lepiej jest założyć, że zdrowa osoba jest chora, i przeprowadzić dokładniejsze badania wykluczające to, niż osoba chora jest uznana za zdrową. W tym przypadku samo accurancy nie wystarcza i należy dążyć do minimalizacji tego rodzaju błędu. Na 42 faktycznie chorych 42 źle zostało zaklasyfikowanych jako zdrowych, czyli model w ogóle nie wykrywa chorych osób.

Zadanie 10

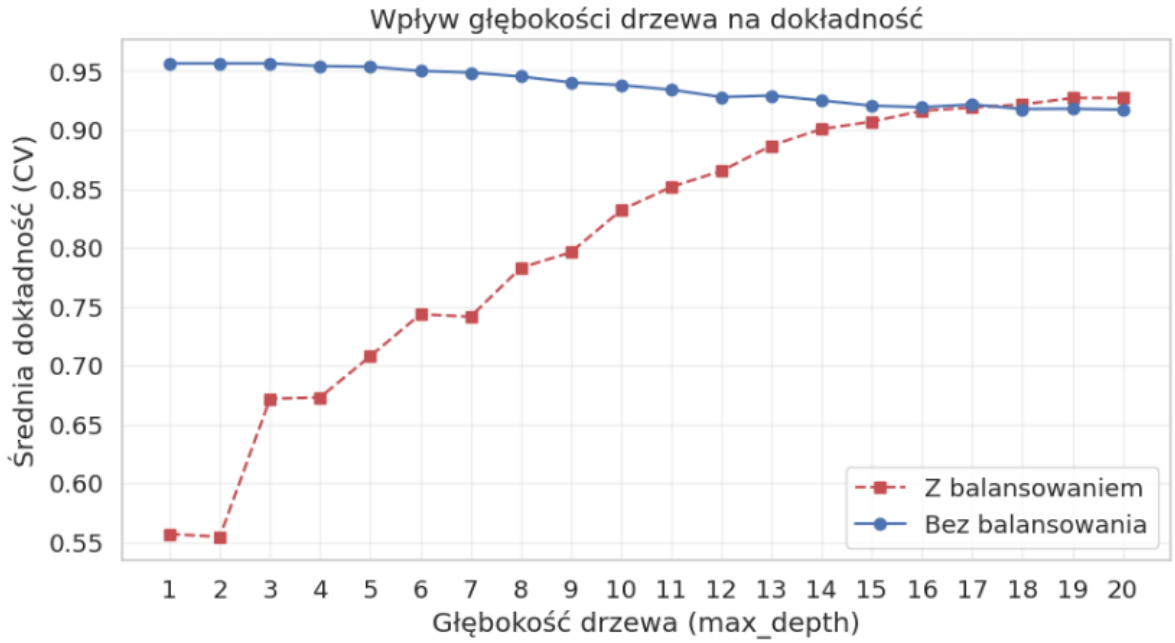
Treść:

10. Powtórz p. 7-9 dla danych „rozmnożonych". Czy osiągnięto poprawę wyników? Które atrybuty są najbardziej istotne (występują w początkowych węzłach)?

Rozwiązanie zadania:

```
#zadanie 7 z balansowaniem
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

depths = range(1, 21)

#Crosswalidacja z balansowaniem
ros = RandomOverSampler(random_state=1)
cv_scores_balanced = []

for d in depths:
    model =
make_pipeline(ros,DecisionTreeClassifier(max_depth=d,random_state=1))
    scores = cross_val_score(model, X_train, y_train, cv=10,
scoring='accuracy')
    cv_scores_balanced.append(scores.mean())

#Wykres
plt.figure(figsize=(10, 5))
plt.plot(depths, cv_scores_balanced, 'rs--', label='Z balansowaniem')
plt.plot(depths, cv_scores, 'bo-', label='Bez balansowania')
plt.xlabel('Głębokość drzewa (max_depth)')
plt.ylabel('Średnia dokładność (CV)')
plt.title('Wpływ głębokości drzewa na dokładność')
plt.xticks(depths)
plt.grid(alpha=0.3)
plt.legend()
plt.show()

#Najlepszy wynik
best_depth_balanced =
depths[cv_scores_balanced.index(max(cv_scores_balanced))]
print(f'\nOptymalna głębokość (z balansowaniem):
{best_depth_balanced}')
print(f'Średnia dokładność CV: {max(cv_scores_balanced):.4f}')

#Wyniki dla wszystkich głębokości
print("\nRozkład dokładności dla różnych głębokości:")
```

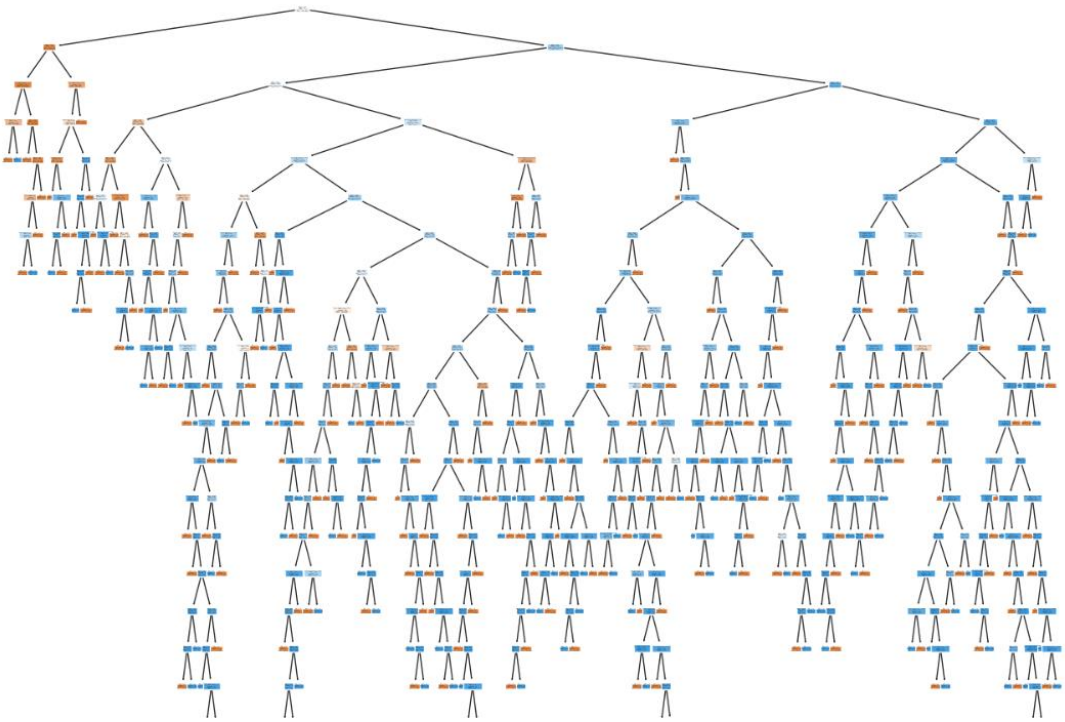
```python
for d, score in zip(depths, cv_scores_balanced):
    print(f"max_depth={d}: {score:.4f}")
```




```
Optymalna głębokość (z balansowaniem): 20
Średnia dokładność CV: 0.9271

Rozkład dokładności dla różnych głębokości:
max_depth=1: 0.5570
max_depth=2: 0.5547
max_depth=3: 0.6718
max_depth=4: 0.6730
max_depth=5: 0.7080
max_depth=6: 0.7436
max_depth=7: 0.7415
max_depth=8: 0.7829
max_depth=9: 0.7962
max_depth=10: 0.8319
max_depth=11: 0.8516
max_depth=12: 0.8652
max_depth=13: 0.8868
max_depth=14: 0.9008
max_depth=15: 0.9071
max_depth=16: 0.9162
max_depth=17: 0.9191
max_depth=18: 0.9217
max_depth=19: 0.9271
max_depth=20: 0.9271
```

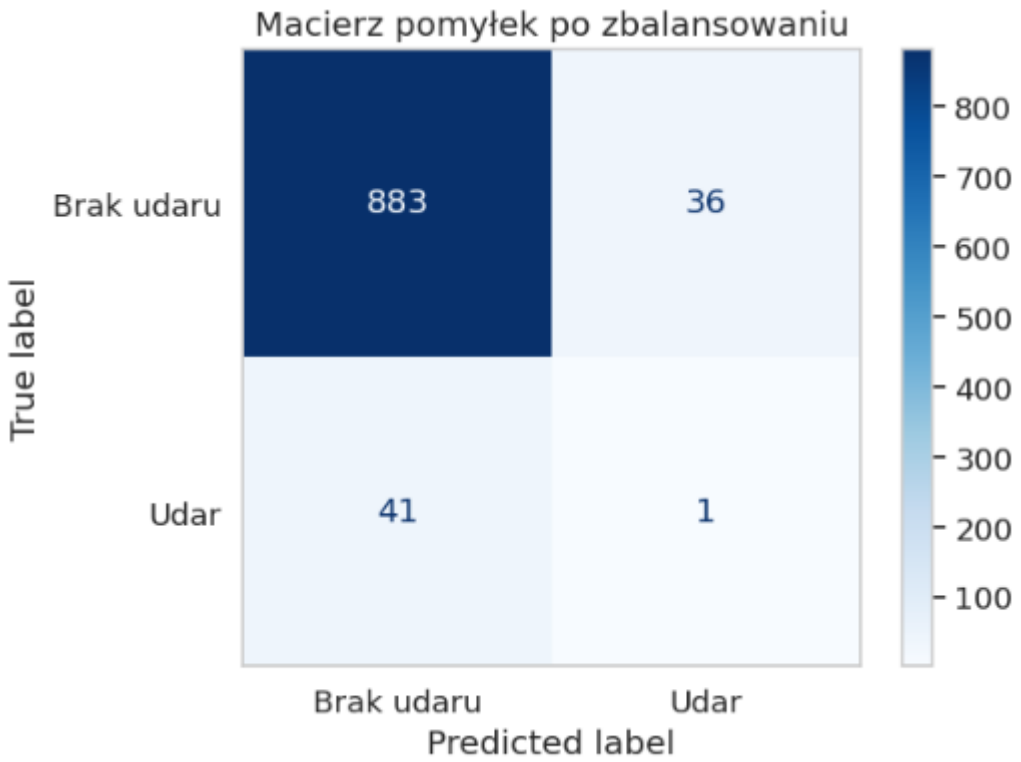
Wybrany został max_depth=20, jest to zdecydowanie zbyt duża wartość, model jest zdecydowanie przeuczony, możnaby ograniczyć ręcznie max_depth do np 4. Dla większej głębokości drzewa wynik z dużym prawdopodobieństwem byłby lepszy, ale dla dużych zbiorów i do wizualizacji jest to bardzo kiepski przypadek.

```
#Zadanie 8 z balansowaniem
from imblearn.over_sampling import RandomOverSampler

#Balansowanie danych
ros = RandomOverSampler(random_state=1)
X_train_bal, y_train_bal = ros.fit_resample(X_train, y_train)
dt_bal =
DecisionTreeClassifier(max_depth=20,random_state=1).fit(X_train_bal,
y_train_bal)

print(f'\nDokładność treningowa: {dt_bal.score(X_train_bal,
y_train_bal):.4f}')
print(f'Dokładność testowa: {dt_bal.score(X_test, y_test):.4f}')

#Drzewo
fig, ax = subplots(figsize=(16, 12))
plot_tree(dt_bal,feature_names=X.columns,class_names=['Brak udaru',
'Udar'],filled=True,ax=ax)
plt.show()

#Reguły decyzyjne
print(export_text(dt_bal,feature_names=list(X.columns),show_weights=Tru
e))
```



Drzewo jest zdecydowanie za mocno rozbudowane, jego układ jest nawet zbyt duży do umieszczenia tutaj, podobnie jak jego reguły.

```
#Zadanie 9 z balansowaniem

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

#Macierz pomyłek
cm = confusion_matrix(y_test, dt_bal.predict(X_test))
plt.figure(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Brak
udaru', 'Udar'])
disp.plot(cmap='Blues', values_format='d')
plt.title('Macierz pomyłek po zbalansowaniu')
plt.grid(False)
plt.show()
```



Wynik nie uległ poprawie, dodatkowo część osób zdrowych klasyfikowana jest jako chorych. Jest to również niezadowalający wynik jak w przypadku braku balansowania.

Zadanie 11

Treść:

11. Używając drzewa regresyjnego dokonaj predykcji wskaźnika `'bmi'` na podstawie pozostałych atrybutów (wliczając atrybut `'stroke'`). W tym celu wykonaj ponownie p. 6-8, odpowiednio je modyfikując.

    Wskazówki:

    - Zamiast `DTC` użyj `DTR` z `criterion='squared_error'`
    - Do oceny modelu zamiast funkcji `accuracy_score()` użyj `mean_squared_error()`
    - W p. 7 do kroswalidacji użyj następującego wywołania funkcji `cross_val_score`:

      ```
      cw[i]= -cross_val_score(dt,
                              X_train2, y_train2,
                              cv=10,
                              scoring='neg_mean_squared_error').mean()
      ```

    - Pamiętaj, że w kroswalidacji szukamy teraz `max_depth`, które minimalizuje błąd, a nie tak jak w klasyfikacji maksymalizuje dokładność.

    Dokonaj wizualizacji używając kodu zamieszczonego poniżej i oceń wyniki.

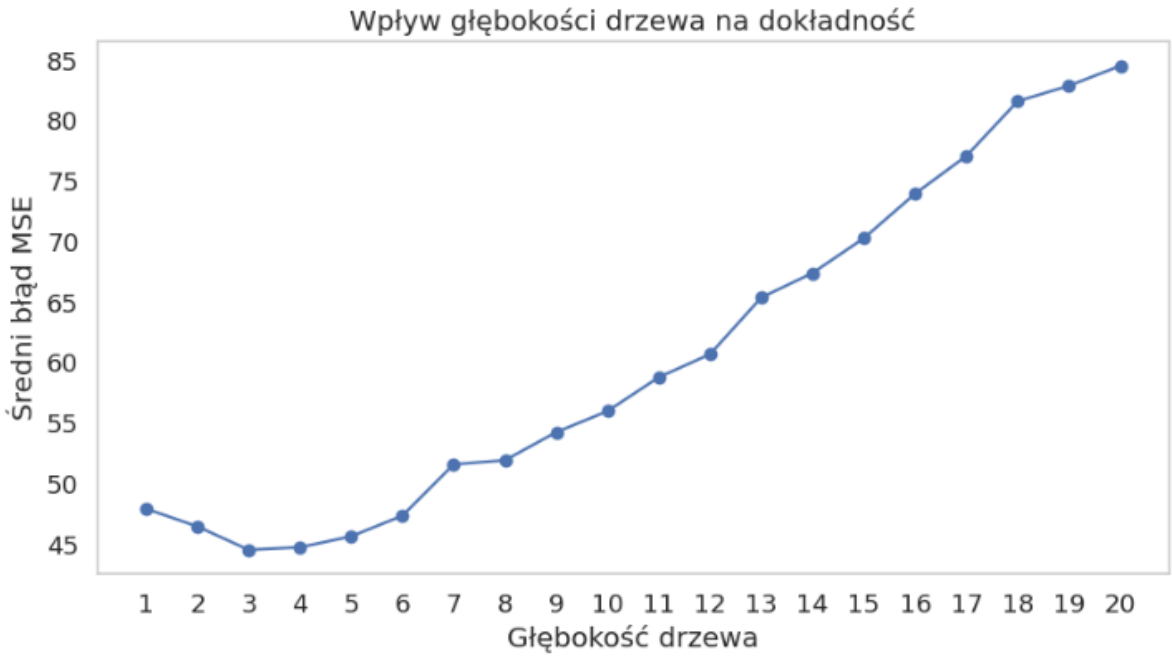
```
ax = subplots(figsize=(6,6))[1]
PredictionErrorDisplay.from_predictions(
    y_true=y_test2,
    y_pred=dt.predict(X_test2),
    #y_true=y_train2,
    #y_pred=dt.predict(X_train2),
    kind="actual_vs_predicted",
    ax=ax,
    random_state=0,
)
```

Rozwiązanie zadania:

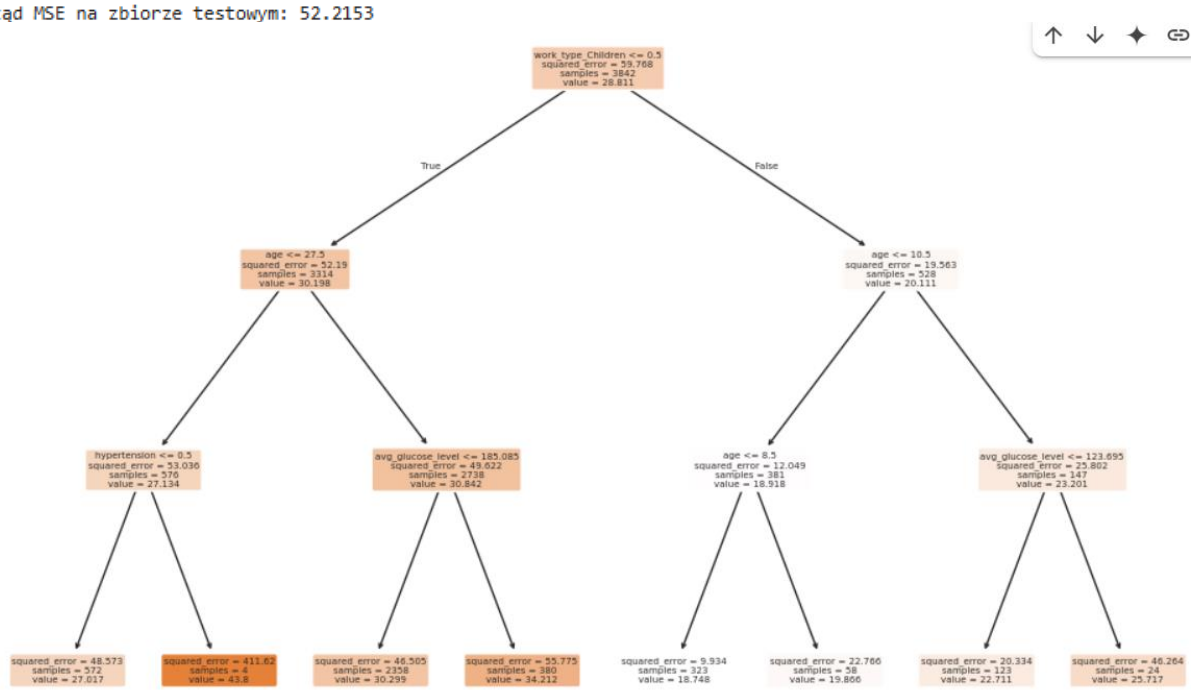
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error

X = pd.get_dummies(data2.drop('bmi', axis=1))
y = data2['bmi']

#Zbiór treningowy i testowy
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=1)

wyniki = []
glebokosci = range(1, 21)

for g in glebokosci:
    model = DecisionTreeRegressor(max_depth=g, random_state=1)
#10-krotna walidacja krzyżowa
    wyniki_mse = -cross_val_score(model, X_train, y_train, cv=10,
scoring='neg_mean_squared_error')
    srednie_mse = wyniki_mse.mean()
    wyniki.append(srednie_mse)

#Wykres z błędem MSE
plt.figure(figsize=(10, 5))
plt.plot(glebokosci, wyniki, 'bo-')
plt.xlabel('Głębokość drzewa')
plt.ylabel('Średni błąd MSE')
plt.title('Wpływ głębokości drzewa na dokładność')
plt.xticks(glebokosci)
plt.grid()
plt.show()

#Najlepsza głębokość
najlepsza_glebokosc = glebokosci[wyniki.index(min(wyniki))]
print(f"Najlepsza głębokość: {najlepsza_glebokosc}")
print(f"Najmniejszy błąd MSE: {min(wyniki):.4f}")

#Trenowanie modelu z nalepszą głębokością
finalny_model = DecisionTreeRegressor(max_depth=najlepsza_glebokosc,
random_state=1)
finalny_model.fit(X_train, y_train)

prognozy = finalny_model.predict(X_test)
mse_test = mean_squared_error(y_test, prognozy)
print(f"Błąd MSE na zbiorze testowym: {mse_test:.4f}")

#Drzewo
plt.figure(figsize=(12, 8))
plot_tree(finalny_model, feature_names=X.columns, filled=True,
rounded=True)
plt.show()
```

Wynik:




```
Najlepsza głębokość: 3
Najmniejszy błąd MSE: 44.5218
Błąd MSE na zbiorze testowym: 52.2153
```



Wniosek:

Klasteryzacja hierarchiczna z complete linkage skutecznie wyodrębniła naturalne grupy genów o podobnych profilach ekspresji. Wyniki pokazują zarówno główne wzorce ekspresji (duże grupy), jak i potencjalnie unikalne profile (małe grupy).

Zadanie 12

Treść:

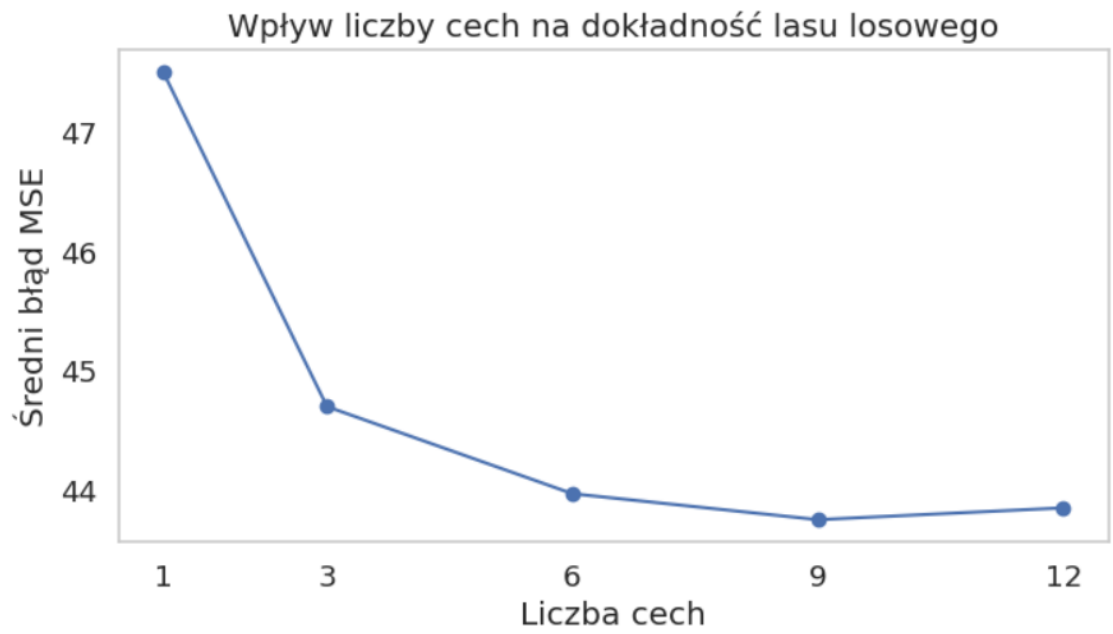
12. Użyj modelu lasu losowego (`RandomForestRegressor`) do rozwiązania problemu regresyjnego postawionego w p. 11. Przyjmij optymalną wartość `max_depth` z p. 11 i domyślne wartości pozostałych hiperparametrów z wyjątkiem `max_features` (liczba atrybutów do przeprowadzenia testu; oznaczony przez m na wykładzie, str. 128 i 129). W kroswalidacji znajdź optymalną wartość hiperparametru `max features`.

Rozwiązanie zadania:

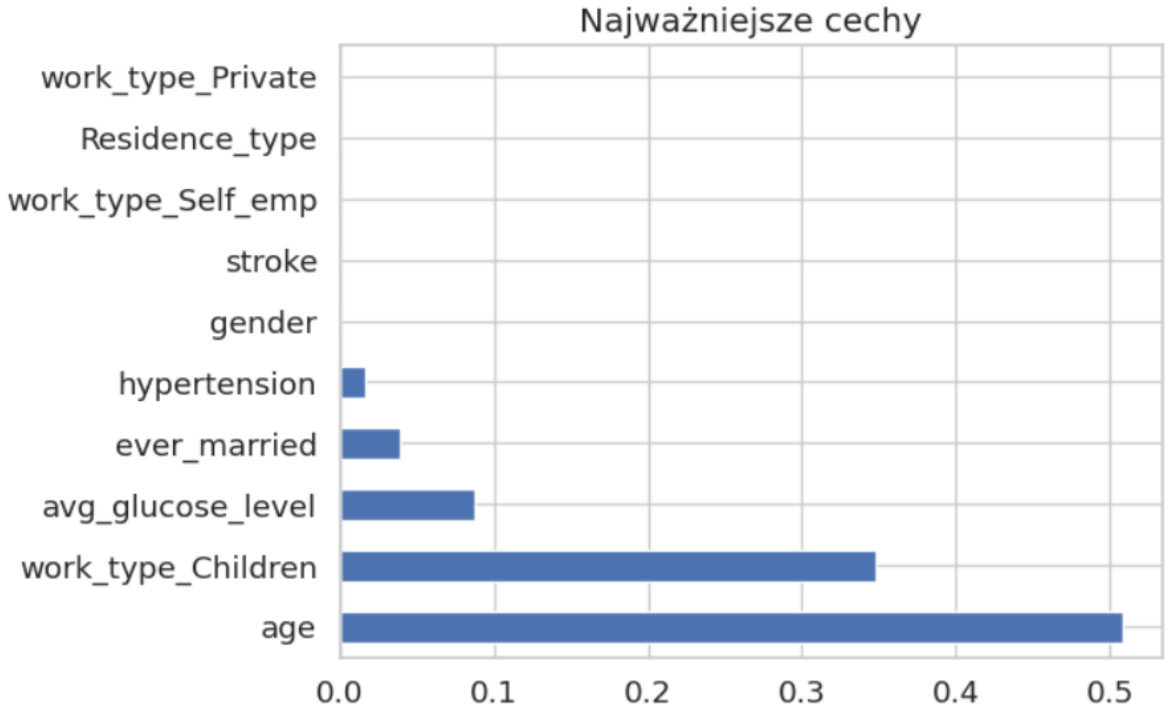
```
from sklearn.ensemble import RandomForestRegressor
import numpy as np

#Liczby cech od 1 do 12
liczba_cech = X.shape[1]
proby_cech = np.unique(np.linspace(1, liczba_cech, 5).astype(int))

wyniki_rf = []
for c in proby_cech:
    model_rf = RandomForestRegressor(n_estimators=100,
max_depth=najlepsza_glebokosc, max_features=c, random_state=1)
    wyniki_mse = -cross_val_score(model_rf, X_train, y_train, cv=10,
scoring='neg_mean_squared_error')
    srednie_mse = wyniki_mse.mean()
    wyniki_rf.append(srednie_mse)

#Wykres
plt.figure(figsize=(8, 4))
plt.plot(proby_cech, wyniki_rf, 'bo-')
plt.xlabel('Liczba cech')
plt.ylabel('Średni błąd MSE')
```

```
plt.title('Wpływ liczby cech na dokładność lasu losowego')
plt.xticks(proby_cech)
plt.grid()
plt.show()

#Najlepsza liczba cech
najlepsza_liczba_cech = proby_cech[np.argmin(wyniki_rf)]
print(f"Najlepsza liczba cech: {najlepsza_liczba_cech}")
print(f"Najmniejszy błąd MSE: {min(wyniki_rf):.4f}")

#Trenowanie modelu
finalny_rf =
RandomForestRegressor(n_estimators=100,max_depth=najlepsza_glebokosc,ma
x_features=najlepsza_liczba_cech,random_state=1)
finalny_rf.fit(X_train, y_train)

prognozy_rf = finalny_rf.predict(X_test)
mse_rf = mean_squared_error(y_test, prognozy_rf)
print(f"Błąd MSE na zbiorze testowym: {mse_rf:.4f}")

#Ważność cech
waznosc_cech = pd.Series(finalny_rf.feature_importances_,
index=X.columns)
waznosc_cech.nlargest(10).plot.barh(title='Najważniejsze cechy')
plt.show()
```

Wynik:




```
Najlepsza liczba cech: 9
Najmniejszy błąd MSE: 43.7573
Błąd MSE na zbiorze testowym: 52.0720
```



Wniosek:

W wyniku otrzymaliśmy, że najoptymalniejszą liczbą cech minimalizującą MSE jest 9. Dodatkowo można zobaczyć z wykresu, że największe znaczenie ma wiek, oraz to, że osoby mają dzieci.

Zadanie 13

Treść:

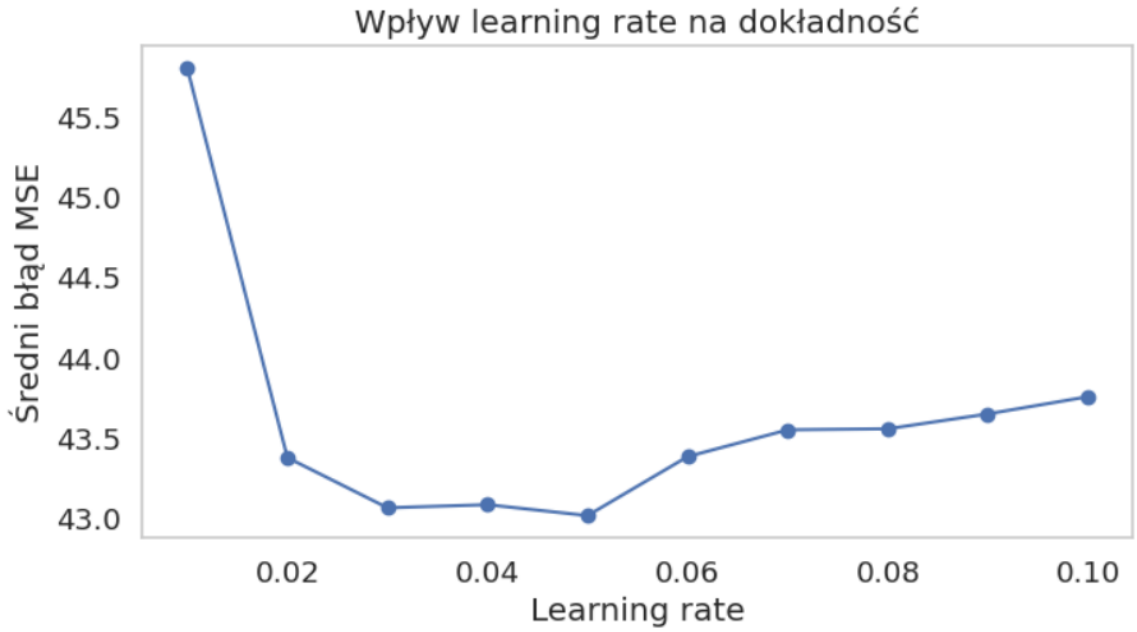
13. Użyj modelu boostingowego opartego na drzewach regresyjnych (`GradientBoostingRegressor`) do rozwiązania problemu regresyjnego postawionego w p. 11. Przyjmij optymalną wartość `max_depth` z p. 11 i domyślne wartości pozostałych hiperparametrów z wyjątkiem `learning_rate` (odpowiednik parametru zawężania λ, patrz wykład str. 130-132). W kroswalidacji znajdź optymalną wartość hiperparametru `learning_rate` w zakresie od 0 do 0.1. Porównaj wyniki otrzymane w p. 11-13.

Rozwiązanie zadania:

```
from sklearn.ensemble import GradientBoostingRegressor

# Sprawdzanie różnych wartości learning rate
wartosci_lr = np.linspace(0.01, 0.1, 10)

wyniki_gb = []
for lr in wartosci_lr:
    model_gb =
GradientBoostingRegressor(max_depth=najlepsza_glebokosc,learning_rate=l
r,n_estimators=100,random_state=1)
    wyniki_mse = -cross_val_score(model_gb, X_train, y_train, cv=10,
scoring='neg_mean_squared_error')
    srednie_mse = wyniki_mse.mean()
    wyniki_gb.append(srednie_mse)

#Wykres
plt.figure(figsize=(8, 4))
plt.plot(wartosci_lr, wyniki_gb, 'bo-')
plt.xlabel('Learning rate')
plt.ylabel('Średni błąd MSE')
plt.title('Wpływ learning rate na dokładność')
plt.grid()
```

```
plt.show()

#Wybór learning rate
najlepszy_lr = wartosci_lr[np.argmin(wyniki_gb)]
print(f"Najlepszy learning rate: {najlepszy_lr:.3f}")
print(f"Najmniejszy błąd MSE: {min(wyniki_gb):.4f}")

#Trenowanie modelu
finalny_gb =
GradientBoostingRegressor(max_depth=najlepsza_glebokosc,learning_rate=n
ajlepszy_lr,n_estimators=100,random_state=1)
finalny_gb.fit(X_train, y_train)

#MSE
prognozy_gb = finalny_gb.predict(X_test)
mse_gb = mean_squared_error(y_test, prognozy_gb)
print(f"Błąd MSE na zbiorze testowym: {mse_gb:.4f}")

#Porównanie modeli
print("\nPorównanie modeli:")
print(f"- Drzewo decyzyjne: {mse_test:.4f}")
print(f"- Las losowy: {mse_rf:.4f}")
print(f"- Gradient Boosting: {mse_gb:.4f}")
```

Wynik:




```
Najlepszy learning rate: 0.050
Najmniejszy błąd MSE: 43.0159
Błąd MSE na zbiorze testowym: 50.8002

Porównanie modeli:
- Drzewo decyzyjne: 52.2153
- Las losowy: 52.0720
- Gradient Boosting: 50.8002
```

Wniosek:

Dla lambdy 0,05 otrzymaliśmy najmniejszy MSE, w porównaniu do pozostałych metod ocecny przy użyciu MSE wynik nie jest znacznie lepszy.

Wnioski końcowe:

Wstępna obróbka danych obejmowała usunięcie zbędnych kolumn, brakujących wartości oraz duplikatów. Dane zostały również przekształcone w formę odpowiednią dla modeli uczenia maszynowego, np. zmienne kategoryczne zakodowano jako binarne lub liczbowo. Mimo to, problemem okazało się silne niezbalansowanie klas w zbiorze danych, gdzie większość przypadków dotyczyła osób zdrowych (brak udaru), co wpłynęło na późniejsze wyniki modeli.

Wizualizacja danych pozwoliła na wstępne zidentyfikowanie zależności między zmiennymi a występowaniem udaru. Najsilniejszą korelację z udarem wykazały wiek, nadciśnienie oraz średni poziom glukozy. Natomiast zmienne takie jak miejsce zamieszkania czy status palenia nie wykazywały wyraźnych związków z celem predykcyjnym.

Bez balansowania klas: Model osiągał wysoką dokładność (accuracy) na poziomie ~95%, jednak wynikało to głównie z przewagi klasy większościowej. Macierz pomyłek

wykazała, że model nie był w stanie poprawnie klasyfikować przypadków udaru, wszystkie chore osoby zostały błędnie zaklasyfikowane jako zdrowe.

Z balansowaniem klas: Po zastosowaniu balansowania dokładność modelu spadła, ale nieznacznie poprawiła się zdolność do wykrywania przypadków udaru.

Optymalna głębokość drzewa: Wykres krzyżowej walidacji wskazał, że zwiększanie głębokości drzewa nie poprawiało znacząco dokładności, a wręcz prowadziło do przeuczenia.

Las losowy: Minimalizacja błędu MSE została osiągnięta przy użyciu 9 cech. Najważniejszymi zmiennymi okazały się wiek oraz posiadanie dzieci.

Gradient Boosting: Najlepszy wynik (najmniejszy MSE) uzyskano dla learning rate równego 0.05. Metoda ta nieznacznie przewyższyła las losowy i drzewo decyzyjne pod względem dokładności.

Porównanie modeli: Gradient Boosting okazał się nieco lepszy od lasu losowego i drzewa decyzyjnego, ale różnice w błędzie MSE nie były znaczące. Wszystkie modele miały trudności z dokładnym przewidywaniem wartości BMI, co sugeruje, że zmienne objaśniające nie są silnie powiązane z tą cechą.

Ćwiczenie pokazało, że kluczowym wyzwaniem w analizie danych medycznych jest niezbalansowanie klas, które znacząco wpływa na skuteczność modeli. W przypadku drzew decyzyjnych proste modele okazały się niewystarczające, a bardziej zaawansowane metody np. boosting przyniosły jedynie marginalne poprawy.